

Catalogizer

Advanced multi-protocol media collection management — detect, catalog, and enrich everything you own.

Summary

Catalogizer is a self-hostable media collection management system that automatically detects, categorizes, and organizes media across SMB, FTP, NFS, WebDAV, and local filesystems, with real-time monitoring, encrypted storage, external metadata enrichment, and a modern React UI backed by a high-performance Go API.

Short description

A production-grade, multi-protocol media library manager. A Go/Gin REST API detects 50+ media types across SMB/FTP/NFS/WebDAV/local sources, enriches from TMDB/IMDB/MusicBrainz/Steam and more, and serves a real-time React web app over an encrypted SQLCipher database.

Long description

Most media managers ask you to surrender first: consolidate everything onto one disk, in one format, of one type, and only then will they help. Catalogizer starts from the opposite premise — your collection already lives where it lives, spread across NAS shares and protocols that will never agree — and meets it there. It speaks the protocols storage already uses — SMB/CIFS, FTP/FTPS, NFS, WebDAV, and local filesystem — behind a single unified client abstraction, so a Windows share, an FTP archive, and a WebDAV mount all look identical to the layers above and can be mixed, swapped, or retired without touching application code. A Go backend (Gin REST API) continuously watches those sources, detects and classifies 50+ media types (movies, TV, music, games, software, documentaries) as files appear, and enriches each item from a stack of external providers — TMDB, IMDB, TVDB, MusicBrainz, Spotify, Steam and more — so a bare

filename becomes a fully-attributed catalog entry with artwork, cast, and metadata. Results stream to a TypeScript React frontend over WebSockets, so the library updates live as ingestion happens rather than on a manual refresh, and every byte of metadata is persisted in an encrypted SQLCipher database gated by JWT-based, role-based authentication.

Where most catalogers quietly fall over the moment a share drops, Catalogizer is engineered to stay useful through the outage. A temporary SMB failure is absorbed by exponential-backoff reconnection, a circuit-breaker that stops hammering a dead host, continuous health monitoring, and an offline metadata cache that keeps answering user requests from the last known-good state — the difference between “the whole app is down because one NAS rebooted” and “one source is degraded and everything else works.” Beyond cataloging, it doubles as an operational tool for a collection: analytics on growth trends and quality/version tracking, professional PDF report generation, a PDF-to-image/text/HTML conversion service, favorites export/import (JSON/CSV), and cloud sync to S3, Google Cloud Storage, or local folders. And it is not a monolith that happens to be big — it is deliberately assembled from 21 reusable `digital.vasic.*` Go submodules plus TypeScript client packages, each independently tested and versioned, so the same battle-hardened auth, filesystem, streaming, and observability pieces that run Catalogizer also power the wider product family. QA is not self-reported: the Challenges framework and HelixQA hold every advertised capability to anti-bluff, evidence-backed verification.

Why we built it

Existing media managers assume a single storage backend and a single media type. Real collections live across many NAS shares and protocols, degrade when a share drops, and mix movies, music, games, and software. Catalogizer was built to treat all protocols equally, survive flaky network storage, and provide one authoritative, enriched, encrypted catalog over everything.

Why it's a game-changer

It collapses into one self-hostable, encrypted package what normally takes a stack of separate tools: protocol-agnostic ingestion that treats every storage backend equally, resilience that keeps the catalog live through storage outages instead of crashing with them, and rich multi-provider enrichment that turns raw files into a browsable, attributed library. The payoff of the extracted-module architecture is compounding: a hardening fix to the filesystem client or a new provider plugin lands once and lifts every consumer, so Catalogizer gets steadily better as the fleet

around it does. It is, in short, the difference between a media index and a media *system* — one you own, one that survives flaky infrastructure, and one whose internals are proven rather than promised.

What's innovative

- Unified multi-protocol filesystem client (SMB/FTP/NFS/WebDAV/local) behind one interface.
- Offline cache + circuit breaker so the catalog stays usable during storage outages.
- Full extraction into 21 reusable `digital.vasic.*` Go submodules and TS client modules.
- Encrypted-at-rest catalog (SQLCipher) with real-time WebSocket sync to the UI.
- Evidence-backed QA via the Challenges framework and HelixQA integration.

Challenges & solutions

- **Flaky network storage:** solved with exponential backoff, circuit breaker, health checks, and an eviction-policy offline cache that serves cached metadata when sources are unreachable.
- **Protocol heterogeneity:** solved by abstracting every protocol behind a common `digital.vasic.filesystem` client so higher layers are protocol-unaware.
- **Data security:** solved with SQLCipher encryption at rest plus JWT/RBAC auth and request sanitization middleware.
- **Maintainability at scale:** solved by extracting all generic logic into independently tested submodules rather than a monolith.

Tech stack (why + how)

- **Go + Gin** — high-performance REST API core (`catalog-api`); chosen for concurrency and throughput on continuous monitoring workloads.
- **TypeScript + React + Tailwind (Vite)** — responsive catalog-web UI with real-time updates.
- **WebSockets** — live data synchronization between backend hub and UI.
- **SQLCipher (encrypted SQLite)** — encrypted-at-rest metadata store; dual SQLite/PostgreSQL support via `digital.vasic.database`.
- **SMB/FTP/NFS/WebDAV clients** — multi-protocol ingestion via `digital.vasic.filesystem`.

- **External metadata APIs (TMDB, IMDB, TVDB, MusicBrainz, Spotify, Steam)** — provider plugins for enrichment.
- **Prometheus + OpenTelemetry** — metrics/tracing via `digital.vasic.observability`.
- **Docker / builder container** — reproducible builds (Tauri/Rust routed through `catalogizer-builder`).
- **Redis** — caching/rate limiting via `digital.vasic.cache / ratelimiter`.
- **S3 / Google Cloud Storage** — cloud sync and checkpoint storage.